

LoveCode

Pablo Greus

```
mirror_mod = modifier_ob.  
set mirror object to mirror.  
mirror_mod.mirror_object
```

```
operation == "MIRROR_X":  
    mirror_mod.use_x = True  
    mirror_mod.use_y = False  
    mirror_mod.use_z = False  
operation == "MIRROR_Y":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = True  
    mirror_mod.use_z = False  
operation == "MIRROR_Z":  
    mirror_mod.use_x = False  
    mirror_mod.use_y = False  
    mirror_mod.use_z = True
```

```
selection at the end -add  
mirror_ob.select= 1  
modifier_ob.select=1  
context.scene.objects.active  
("Selected" + str(modifier_ob.  
mirror_ob.select = 0  
= bpy.context.selected_object  
data.objects[one.name].select  
print("please select exactly
```

```
-- OPERATOR CLASSES --
```

```
types.Operator):  
    X mirror to the selected  
object.mirror_mirror_x"  
mirror X"
```

```
context):  
context.active_object is not
```

Que es LoveCode?

- LoveCode es una aplicación web de tipo matching Inspirada en la mecánica de deslizamiento de tarjetas popularizada por aplicaciones de citas

¿Cómo funciona?

1



Crear cuenta

Nombre, correo, contraseña y bio. Seleccionas tus tecnologías favoritas.

2



Iniciar sesión

Accedes con tu correo y contraseña. Tu sesión queda guardada.

3



Explorar perfiles

Ves tarjetas de otros devs con su nombre, bio y tecnologías. Desliza o usa botones.

4



Dar like

Si te gusta un perfil, le das like. Si la otra persona también te lo da...

5



¡Es un Match!

¡Conexión establecida! Aparece en tu página de matches.

Las pantallas de la aplicación



Inicio

Portada con botones para iniciar sesión o registrarse



Login

Formulario de correo y contraseña



Registro

Formulario completo con selector de tecnologías en chips



Dashboard

Pila de tarjetas de perfiles. Swipe o botones ✕ ❤️ ↺



Matches

Lista de todos los devs con quienes hiciste match



Bienvenido a CodeCrush

Donde los devs encuentran su
match perfecto ❤️

→ Iniciar Sesión

✦ Crear cuenta

// Tu próximo commit podría ser al amor 💻

Las pantallas de la aplicación



Inicio

Portada con botones para iniciar sesión o registrarse



Login

Formulario de correo y contraseña



Registro

Formulario completo con selector de tecnologías en chips



Dashboard

Pila de tarjetas de perfiles. Swipe o botones ✕ ❤️ ↺



Matches

Lista de todos los devs con quienes hiciste match

[← Volver al inicio](#)

Iniciar Sesión

// Conecta y encuentra tu match

CORREO

 tu@correo.com

CONTRASEÑA

 ••••••••

→ ENTRAR

¿No tienes cuenta? [Regístrate aquí](#)

Las pantallas de la aplicación



Inicio

Portada con botones para iniciar sesión o registrarse



Login

Formulario de correo y contraseña



Registro

Formulario completo con selector de tecnologías en chips



Dashboard

Pila de tarjetas de perfiles. Swipe o botones ✕ ❤️ ↺



Matches

Lista de todos los devs con quienes hiciste match

[← Volver al inicio](#)

Crear cuenta

// Tu historia de amor en código empieza aquí

NOMBRE

 Tu nombre

CORREO ELECTRÓNICO

 tu@correo.com

CONTRASEÑA

 ••••••••

BIO (opcional)

Cuéntanos algo sobre ti... ¿qué stack usas? ¿tabs o spaces?

TUS TECNOLOGÍAS

JavaScript

Python

Java

TypeScript

SQL

React

CSS

C#

✦ REGISTRARSE

¿Ya tienes cuenta? [Inicia sesión](#)

Las pantallas de la aplicación



Inicio

Portada con botones para iniciar sesión o registrarse



Login

Formulario de correo y contraseña



Registro

Formulario completo con selector de tecnologías en chips



Dashboard

Pila de tarjetas de perfiles. Swipe o botones ✕ ❤️ ↺



Matches

Lista de todos los devs con quienes hiciste match

← Volver al inicio

0 / 3  Matches 

Encuentra tu match

// Desliza y conecta con devs afines



María López

@maria28

@carlos

// sin tecnologías
// sin tecnologías
profesional en excel

Java

SQL

ID: 2

carlos1



// Tu próximo commit podría ser al amor 

Las pantallas de la aplicación



Inicio

Portada con botones para iniciar sesión o registrarse



Login

Formulario de correo y contraseña



Registro

Formulario completo con selector de tecnologías en chips



Dashboard

Pila de tarjetas de perfiles. Swipe o botones ✕ ❤️ ↺



Matches

Lista de todos los devs con quienes hiciste match

← Volver

1 match

Tus matches

// Devs que también te dieron like



Carlos Gómez
@carlos

22 may 2026

// El amor también se compila 

Estructura del código Java

El backend está organizado en 3 capas. Cada una tiene una responsabilidad clara:



1. Controller

Recibe las peticiones del navegador

Es la "puerta de entrada" de la API. Escucha rutas como /api/login o /api/likes y decide qué hacer con cada petición.

Archivos:

UserController · LikeController ·
MachController



2. Modelo (POJO)

Representa los datos del sistema

Clases simples que modelan los objetos del negocio: un Usuario, un Like, un Match, una Tecnología. Sin lógica, solo atributos.

Archivos:

Usuario · Like · Mach · Tecnologia



3. DAO (Data Access Object)

Habla directamente con la base de datos

Contiene las consultas SQL. Inserta, busca y actualiza datos. El Controller le pide datos y el DAO se los devuelve.

Archivos:

UsuarioDAO · LikeDAO · MachDAO ·
TecnologiaDAO

Endpoints de la API REST

Base URL: `http://localhost:8080/api`

MÉTODO	RUTA	QUÉ HACE	USADO EN
POST	<code>/registro</code>	Crea una cuenta nueva. Recibe nombre, correo, contraseña, bio y tecnologías. Devuelve el id del usuario creado.	<code>Formregistrer.js</code>
POST	<code>/login</code>	Comprueba correo y contraseña. Si son correctos devuelve los datos del usuario (id, nombre, correo) para guardarlos en localStorage.	<code>Formlogin.js</code>
GET	<code>/usuarios</code>	Devuelve la lista completa de usuarios con sus tecnologías. El Dashboard la usa para cargar las tarjetas de perfiles.	<code>Dashboardfuncion.js</code>
POST	<code>/likes</code>	Registra que el usuario A le da like al usuario B. Si B ya le había dado like a A, la respuesta incluye <code>match: true</code> .	<code>Dashboardfuncion.js</code>
GET	<code>/matches/{idUserario}</code>	Devuelve todos los matches de un usuario con los datos completos del otro dev (nombre, bio, tecnologías y fecha del match).	<code>Machesfuncion.js</code>
GET	<code>/tecnologias</code>	Lista todas las tecnologías disponibles en el catálogo.	<code>TecnologiaController</code>
POST	<code>/tecnologias/vincular</code>	Asigna tecnologías a un usuario. Borra las anteriores y vincula las nuevas. Se llama tras el registro.	<code>UsuarioDAO</code>

¿Cómo funcionan los Likes y Matches?



NO X

El sistema comprueba los dos sentidos:

Ana → Luis + Luis → Ana = MATCH automático vía Trigger en la base de datos o lógica en Java (LikeDAO)

Los matches se guardan con fecha y evitan duplicados gracias a la clave UNIQUE (id_usuario1, id_usuario2)

Procedimientos almacenados y Trigger

⚙️ Procedimientos Almacenados

Son bloques de código SQL guardados en la base de datos que se pueden llamar por nombre. Se ejecutan directamente en MySQL, sin pasar por Java.

CargarUsuario(nombre, correo, bio)

Valida que nombre y correo no estén vacíos ni repetidos y luego inserta el usuario. Devuelve la fila creada.

```
IF correo ya existe → ERROR
INSERT INTO usuario(...)
SELECT * WHERE id = LAST_INSERT_ID()
```

BorrarUsuario(id_usuario)

Comprueba que el usuario exista antes de borrarlo. Si no existe lanza un error personalizado.

```
IF NOT EXISTS → SIGNAL SQLSTATE
DELETE FROM usuario WHERE id = p_id
```

ObtenerMatches(id_usuario)

Devuelve todos los matches de un usuario con los datos completos del otro dev (nombre, correo, bio, tecnologías comunes).

```
SELECT u.nombre, tecnologias_comunes
FROM Matches JOIN usuario...
WHERE id_usuario1=? OR id_usuario2=?
```

⚡ Trigger: after_like_insert

Cada vez que alguien da like, comprueba si el otro usuario ya le había dado like antes. Si es así, crea el match automáticamente en la tabla Matches, sin que Java tenga que hacer nada.

after_like_insert (SQL)

```
AFTER INSERT ON Likes
FOR EACH ROW BEGIN

-- ¿El receptor ya nos dio like?
IF EXISTS (SELECT 1 FROM Likes
  WHERE id_emisor = NEW.id_receptor
  AND id_receptor = NEW.id_emisor)
THEN
  -- Evita duplicados en Matches
  IF NOT EXISTS (SELECT 1 FROM Matches
    WHERE id_u1=LEAST(e,r)
    AND id_u2=GREATEST(e,r))
  THEN
    INSERT INTO Matches(...)
    VALUES(LEAST(...), GREATEST(...))
  END IF;
END IF;
END
```

🔑 LEAST / GREATEST garantizan que (u1 < u2) → no hay (Ana,Luis) y (Luis,Ana) a la vez

Infraestructura y despliegue



La base de datos corre en un contenedor Docker.

Hay un servidor nginx levantando la web

```
docker run --name mi-servidor-nginx -p
5500:80 -v
/ruta/de/tu/carpeta:/usr/share/nginx/html:ro
-d nginx
```

```
services:
  > Run Service
  mysql:
    image: mysql:8.0
    container_name: LoveCodeMySQL
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: LoveCode
      MYSQL_USER: admin
      MYSQL_PASSWORD: admin
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
volumes:
  mysql_data: You, last week • actual
```